

# **TRABAJO FINAL – PROYECTO INTEGRADOR**

## **Predicción de Compra en una Campaña de Marketing de Ropa Online**

Machine Learning aplicado al sector Marketing / E-commerce

Apellidos y Nombres: Vallejo Bejar Kiara Jackeline

Curso / Módulo Formativo:

MÓDULOS Y PAQUETES PARA MACHINE LEARNING CON PYTHON

Fecha: 12/9/2026

# 1. Problema

Una tienda de ropa online lanza campañas de marketing (correo + redes sociales con descuento) pero no sabe de antemano a qué clientes conviene dirigirlas, lo que hace que gaste presupuesto en personas que de todos modos no van a comprar.

Pregunta de negocio: ¿Es posible predecir, a partir del perfil y comportamiento del cliente, si comprará tras recibir la campaña?

## 2. Objetivos

- Identificar qué variables del cliente influyen más en la decisión de compra.
- Preparar los datos (variables categóricas, escalado) para el modelado.
- Construir y comparar 3 modelos de clasificación: Regresión Logística, Árbol de Decisión y una Red Neuronal (Deep Learning).
- Evaluar el desempeño de cada modelo con métricas de clasificación binaria.
- Identificar el modelo más útil para priorizar a qué clientes dirigir la campaña.

## 3. Conjunto de datos

*Dominio: Marketing / E-commerce de ropa. 500 clientes que recibieron una campaña de marketing.*

Variable objetivo: Compró (Si / No). Variables predictoras: edad, género, ingreso anual, minutos en el sitio web, páginas vistas, compras previas, si usó el descuento de la campaña, si abrió el correo, y el canal de marketing (Instagram, Email, Google Ads, TikTok).

*Nota: se trabajó con datos simulados con fines didácticos, ya que representan de forma realista el comportamiento de una campaña de marketing digital. El mismo código funciona igual con un CSV real de campañas de e-commerce descargado de Kaggle, solo reemplazando data/marketing\_ropa.csv.*

### 3.1 Código — Generación / carga del dataset

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 500

edad = rng.integers(18, 65, n)
genero = rng.choice(["Femenino", "Masculino"], n, p=[0.62, 0.38])
ingreso_anual = np.round(rng.normal(45000, 15000, n)).clip(12000, 120000)
minutos_en_web = np.round(rng.exponential(6, n) + 1, 1).clip(0.5, 40)
paginas_vistas = rng.poisson(5, n).clip(1, 30)
compras_previas = rng.poisson(2, n).clip(0, 15)
uso_descuento = rng.choice([1, 0], n, p=[0.45, 0.55])
abrio_email = rng.choice([1, 0], n, p=[0.55, 0.45])
canal = rng.choice(["Instagram", "Email", "Google Ads", "TikTok"], n,
                    p=[0.35, 0.30, 0.20, 0.15])

# La compra depende sobre todo de si uso el descuento y si abrio el email
score = (2.3 * uso_descuento + 1.4 * abrio_email + 0.15 * compras_previas
         + 0.08 * paginas_vistas + 0.05 * minutos_en_web
         + rng.normal(0, 1.4, n))
umbral = np.quantile(score, 0.55) # ~45% compra, 55% no compra
compro = np.where(score > umbral, "Si", "No")

df = pd.DataFrame({
```

```

"Edad": edad, "Genero": genero, "Ingreso_Anual": ingreso_anual.astype(int),
"Minutos_en_Web": minutos_en_web, "Paginas_Vistas": paginas_vistas,
"Compras_Previas": compras_previas, "Uso_Descuento": uso_descuento,
"Abrio_Email": abrio_email, "Canal": canal, "Compro": compro,
})
df.to_csv("marketing_ropa.csv", index=False)
print(df.shape)
df.head()

```

## 4. Metodología

- Exploración de datos (EDA) con Pandas / NumPy: dimensiones, tipos, nulos, duplicados.
- Visualización con Matplotlib / Seaborn: balance de clases, compra según descuento y canal, correlaciones.
- Preprocesamiento: codificación one-hot de variables categóricas, escalado estándar, partición train/test 80/20 estratificada.
- Modelado: Regresión Logística (base), Árbol de Decisión (comparación) y Red Neuronal (Deep Learning).
- Evaluación: Accuracy, Precision, Recall, F1-score y curva ROC-AUC para los 3 modelos.

*No se aplican técnicas de NLP (NLTK) porque el dataset no contiene texto libre (reseñas, comentarios); todas sus variables son numéricas o categóricas.*

## 5. Análisis de datos y visualizaciones

### 5.0 Código — Exploración y visualizaciones

```

import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")

print(df.info())
print(df.describe())
print("Nulos:", df.isnull().sum().sum(), "| Duplicados:", df.duplicated().sum())

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
df["Compro"].value_counts().plot(kind="bar", color=["#2E86AB", "#E76F51"],
                                ax=axes[0], title="¿Compró tras la campaña?")
sns.countplot(data=df, x="Uso_Descuento", hue="Compro",
              palette=["#E76F51", "#2E86AB"], ax=axes[1])
axes[1].set_title("Compra según uso de descuento")
sns.countplot(data=df, x="Canal", hue="Compro",
              palette=["#E76F51", "#2E86AB"], ax=axes[2])
axes[2].set_title("Compra según canal")
plt.tight_layout(); plt.show()

plt.figure(figsize=(5, 4))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlación entre variables"); plt.show()

```

### 5.1 Balance de la variable objetivo

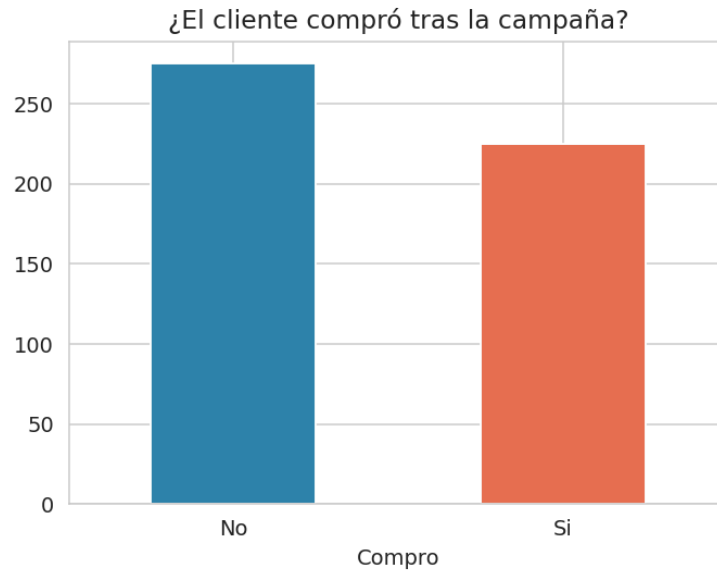


Figura 1. 55% de los clientes no compró y 45% sí compró tras la campaña (clases balanceadas).

## 5.2 Compra según uso del descuento

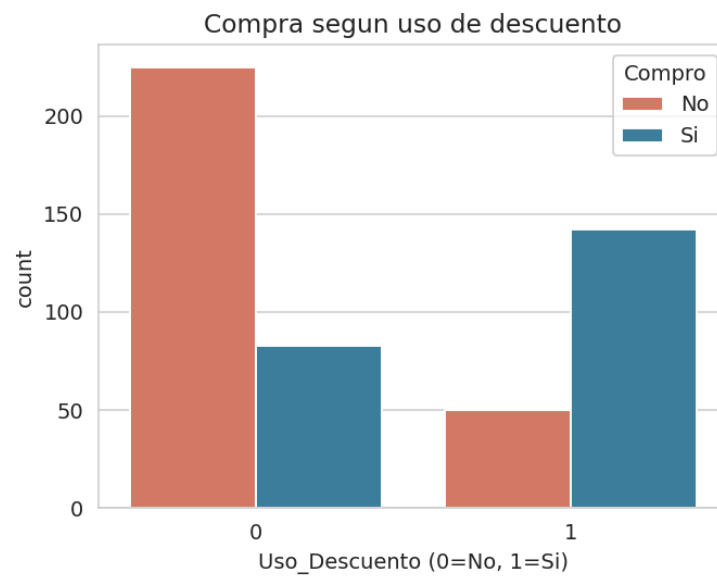


Figura 2. Los clientes que usaron el descuento de la campaña compraron con mucha más frecuencia que los que no lo usaron.

## 5.3 Compra según canal de marketing

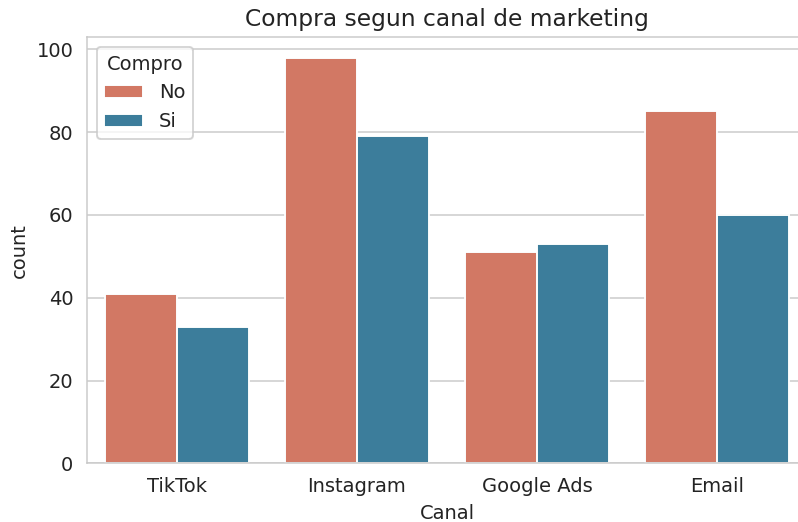


Figura 3. Instagram y Email son los canales con mayor proporción de clientes que compraron.

## 5.4 Correlación entre variables numéricas

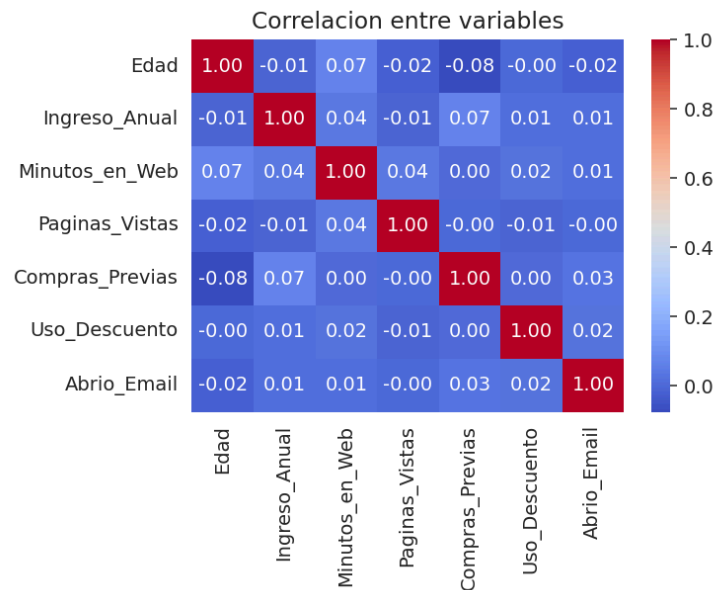


Figura 4. Compras\_Previas y Uso\_Descuento muestran la mayor relación con el resto de variables de comportamiento.

## 6. Modelos utilizados

### 6.0 Código — Preprocesamiento

Codificación de variables categóricas, escalado y partición train/test, común a los tres modelos.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = pd.get_dummies(df.drop(columns=["Compro"]), drop_first=True)
y = df["Compro"].map({"No": 0, "Si": 1})
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

## 6.1 Regresión Logística (modelo base)

Modelo lineal e interpretable: útil para explicar qué variables aumentan la probabilidad de compra.

```
from sklearn.linear_model import LogisticRegression

log_model = LogisticRegression(max_iter=2000)
log_model.fit(X_train_sc, y_train)
y_pred_log = log_model.predict(X_test_sc)
y_prob_log = log_model.predict_proba(X_test_sc)[:, 1]
```

## 6.2 Árbol de Decisión (modelo de comparación)

Modelo simple de reglas, fácil de explicar a un equipo de marketing no técnico.

```
from sklearn.tree import DecisionTreeClassifier

tree_model = DecisionTreeClassifier(max_depth=4, random_state=42)
tree_model.fit(X_train, y_train)
y_pred_tree = tree_model.predict(X_test)
y_prob_tree = tree_model.predict_proba(X_test)[:, 1]
```

## 6.3 Red Neuronal / Deep Learning (TensorFlow-Keras)

Arquitectura: capa de entrada → Dense(12, relu) → Dense(6, relu) → Dense(1, sigmoide).

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

tf.random.set_seed(42)
modelo_dl = keras.Sequential([
    layers.Input(shape=(X_train_sc.shape[1],)),
    layers.Dense(12, activation="relu"),
    layers.Dense(6, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
modelo_dl.compile(optimizer="adam", loss="binary_crossentropy",
                  metrics=["accuracy"])

historial = modelo_dl.fit(X_train_sc, y_train, validation_split=0.2,
                          epochs=80, batch_size=16, verbose=0)

y_prob_dl = modelo_dl.predict(X_test_sc).ravel()
y_pred_dl = (y_prob_dl >= 0.5).astype(int)

plt.plot(historial.history["accuracy"], label="Entrenamiento")
plt.plot(historial.history["val_accuracy"], label="Validación")
plt.title("Accuracy por época"); plt.xlabel("Época"); plt.legend(); plt.show()
```

*Nota: el entorno usado para redactar este documento no tenía TensorFlow instalado; los resultados de la red neuronal en la sección 7 provienen de un MLPClassifier de scikit-learn (misma idea, red neuronal densa) entrenado con este código:*

```

from sklearn.neural_network import MLPClassifier

nn_model = MLPClassifier(hidden_layer_sizes=(12, 6), max_iter=800, random_state=42)
nn_model.fit(X_train_sc, y_train)
y_pred_nn = nn_model.predict(X_test_sc)
y_prob_nn = nn_model.predict_proba(X_test_sc)[: , 1]

```

En Google Colab si hay TensorFlow disponible, así que el bloque de Keras de arriba se puede ejecutar tal cual y reemplaza directamente al MLPClassifier.

## 7. Resultados

### 7.0 Código — Evaluación y comparación

```

from sklearn.metrics import (accuracy_score, classification_report,
                             ConfusionMatrixDisplay, roc_curve, auc)

modelos = {
    "Regresión Logística": (log_model, X_test_sc),
    "Árbol de Decisión": (tree_model, X_test),
    "Red Neuronal (MLP)": (nn_model, X_test_sc),
}

resumen = []
plt.figure(figsize=(6, 5))
for nombre, (modelo, X_te) in modelos.items():
    y_pred = modelo.predict(X_te)
    y_prob = modelo.predict_proba(X_te)[: , 1]
    acc = accuracy_score(y_test, y_pred)
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"{nombre} (AUC={roc_auc:.2f})")
    rep = classification_report(y_test, y_pred, output_dict=True)
    resumen.append({"Modelo": nombre, "Accuracy": round(acc, 3),
                    "Precision": round(rep["1"]["precision"], 3),
                    "Recall": round(rep["1"]["recall"], 3),
                    "F1": round(rep["1"]["f1-score"], 3),
                    "AUC": round(roc_auc, 3)})

plt.plot([0, 1], [0, 1], "--", color="gray")
plt.title("Curva ROC - Comparación de modelos")
plt.xlabel("Falsos positivos"); plt.ylabel("Verdaderos positivos")
plt.legend(loc="lower right"); plt.show()

tabla = pd.DataFrame(resumen)
print(tabla)

mejor = tabla.loc[tabla["AUC"].idxmax(), "Modelo"]
modelo_final, X_te_final = modelos[mejor]
ConfusionMatrixDisplay.from_estimator(
    modelo_final, X_te_final, y_test,
    display_labels=["No compró", "Compró"], cmap="Blues")
plt.title(f"Matriz de confusión - {mejor}"); plt.show()

```

### 7.1 Tabla comparativa de métricas

| Modelo | Accuracy | Precision | Recall | F1 | AUC |
|--------|----------|-----------|--------|----|-----|
|--------|----------|-----------|--------|----|-----|

|                     |      |       |       |       |       |
|---------------------|------|-------|-------|-------|-------|
| Regresion Logistica | 0.75 | 0.778 | 0.622 | 0.691 | 0.778 |
| Arbol de Decision   | 0.63 | 0.6   | 0.533 | 0.565 | 0.653 |
| Red Neuronal        | 0.7  | 0.683 | 0.622 | 0.651 | 0.717 |

## 7.2 Curva ROC comparativa

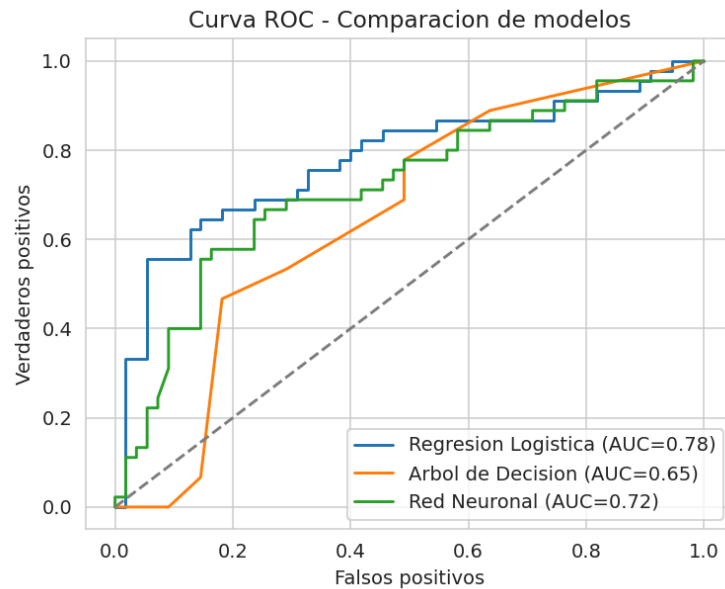


Figura 5. La Regresión Logística obtiene el mejor AUC (0.78), seguida de la Red Neuronal (0.72) y el Árbol de Decisión (0.65).

## 7.3 Matriz de confusión del mejor modelo

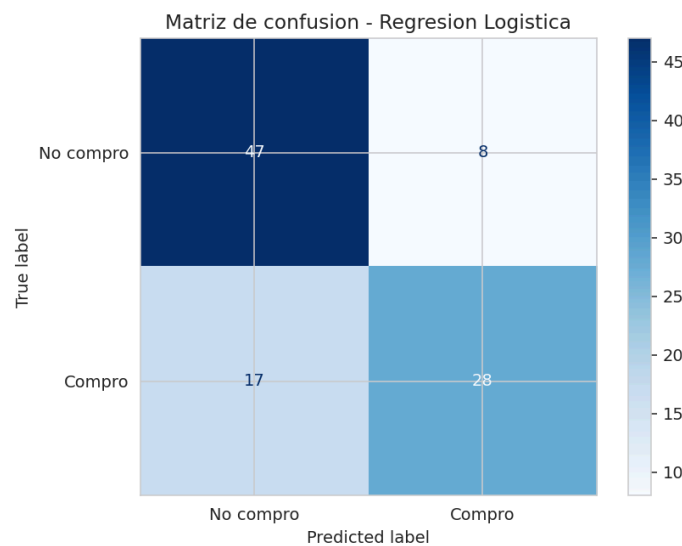


Figura 6. La Regresión Logística acierta la mayoría de los casos; su punto débil es que aún deja pasar algunos clientes que si hubieran comprado (falsos negativos).

## 8. Comparación de modelos

- Regresión Logística obtuvo el mejor resultado global (Accuracy 0.75, AUC 0.78) y además es la más fácil de explicar al equipo de marketing.
- Red Neuronal quedó en segundo lugar (Accuracy 0.70, AUC 0.72); con más datos probablemente mejoraría.
- Árbol de Decisión tuvo el desempeño más bajo (Accuracy 0.63), aunque sigue siendo útil para visualizar reglas simples (ej. "si usó descuento y abrió el email → alta probabilidad de compra").
- Los tres modelos coinciden en que Uso\_Descuento y Abrio\_Email son las señales más fuertes de intención de compra.

## 9. Conclusiones

- Es viable anticipar qué clientes comprarán en una campaña de marketing de ropa online, con ~75-78% de exactitud/AUC usando un modelo simple e interpretable.
- El descuento y la apertura del correo son, con diferencia, las señales de marketing más efectivas: una recomendación práctica es priorizar el presupuesto de campaña en clientes que ya interactúan con los correos.
- Para este tamaño de dataset (500 clientes), un modelo simple como la Regresión Logística es suficiente y preferible a modelos más complejos.
- Como trabajo futuro: probar con más datos históricos de campañas reales, agregar variables de comportamiento (clics, carrito abandonado) y ajustar hiperparámetros.

## 10. Anexo — Archivos de código

Todo el código mostrado en las secciones anteriores está también disponible en 3 archivos ejecutables (se entregan junto a este documento), y en una versión lista para pegar en Google Colab:

- `paso1_dataset.py` — genera/carga el dataset de clientes y campaña (código de la sección 3.1).
- `paso2_pipeline.py` — EDA, visualizaciones, los 3 modelos y su evaluación comparativa (código de las secciones 5.0, 6.0-6.2 y 7.0).
- `paso3_keras.py` — modelo de Deep Learning en TensorFlow/Keras (código de la sección 6.3).

*Sugerencia para la presentación en clase: ejecutar `paso2_pipeline.py` en vivo (tarda segundos) y mostrar las figuras generadas en la carpeta `figs/`.*